TRABAJO FINAL INTEGRADOR METODOLOGÍA II

Sistema de Gestión de Solicitudes Académicas

Profesor:

Integrantes:

Celeste Monsalbe - DNI 45.028.610 - Comisión

Sol Yoon - DNI 92.906.418 - Comisión

Fecha de entrega:08/11/2026

Link Video:

Link Repositorio GitHub:

https://github.com/bks999/tpi-solicitudes.git

UTN
TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

0. Resumen ejecutivo

1. Selección y descripción del sistema base

Dominio

El sistema elegido es un **Sistema de Gestión de Solicitudes Académicas** para una institución universitaria ficticia. Permite centralizar los trámites que los estudiantes presentan ante distintas áreas administrativas (certificados, equivalencias, bajas de materias, mesas de examen especiales y consultas generales) y seguir cada solicitud desde que se registra hasta que se cierra.

El prototipo fue construido específicamente para este trabajo, con asistencia de una herramienta de IA declarada en la sección correspondiente del informe. Todos los datos que contiene son sintéticos.

Alcance

El sistema cubre el registro de solicitudes y de estudiantes, el seguimiento del estado de cada solicitud, la asignación de un responsable según el área que corresponde al trámite, la búsqueda por distintos criterios y la generación de un reporte resumido.

Quedan fuera del alcance:

- la autenticación de usuarios y los permisos,
- la interfaz gráfica,
- el envío de notificaciones,
- la edición o eliminación de solicitudes ya registradas,
- el alta de nuevos responsables desde el sistema (se cargan desde archivo)
- y el uso simultáneo por varios usuarios.

Actores

El **operador administrativo** es el único usuario directo: personal de la institución que carga las solicitudes y las gestiona desde la consola. El **estudiante** es un actor indirecto: origina la solicitud, pero no interactúa con el sistema, sino que su trámite es registrado por el operador. Los **responsables de área** (Bedelía, Secretaría Académica y Departamento de Alumnos) tampoco usan el sistema: se les asignan solicitudes para que las resuelvan.

Funcionalidades

Opción	Funcionalidad	Reglas principales
1	Registrar solicitud	El legajo debe tener al menos 4 caracteres. Si el estudiante no existe, se lo registra con nombre y apellido obligatorios y un email válido. Se eligen tipo (5 opciones) y prioridad (baja, media o alta), y la descripción debe tener al menos 10 caracteres. Si el tipo es baja de materia o examen especial y la descripción menciona “urgente”, la prioridad se eleva a alta automáticamente. Toda solicitud nueva arranca en estado PENDIENTE, con la fecha actual y sin responsable.
2	Cambiar estado	Solo se permiten estas transiciones: PENDIENTE → EN_REVISION (requiere responsable asignado) o RECHAZADA; EN_REVISION → APROBADA o RECHAZADA; APROBADA o RECHAZADA → CERRADA. Una solicitud CERRADA no se puede modificar. Cada cambio queda registrado con su fecha en el historial de la solicitud.
3	Asignar responsable	No se puede asignar responsable a solicitudes CERRADAS o RECHAZADAS. Cada tipo tiene un área esperada: certificados y exámenes especiales → Bedelía; equivalencias → Secretaría Académica; bajas de materia → Departamento de Alumnos. Si el responsable elegido es de otra área, el sistema pide confirmación. Un responsable no puede tener más de 5 solicitudes activas.
4	Buscar / filtrar	Permite buscar por estado, tipo, prioridad, legajo o responsable (incluidas las solicitudes sin asignar), e informa la cantidad de resultados.
5	Generar reporte	Muestra el total de solicitudes, la cantidad por estado, por tipo y por responsable, las solicitudes sin asignar, la prioridad promedio y las de prioridad alta que siguen abiertas. El reporte se muestra en pantalla y se guarda en data/reporte.txt.
6	Listar solicitudes	Muestra todas las solicitudes registradas con su información completa.

Los datos se guardan en tres archivos: estudiantes, responsables y solicitudes. Se cargan al iniciar el programa y se guardan después de cada operación que modifica información y al salir.

Criterio de priorización

Al registrar una solicitud, el operador le asigna una prioridad que indica qué tan pronto debe atenderse: baja (1), media (2) o alta (3). El sistema no aplica este criterio automáticamente, salvo en un caso: si una baja de materia o un examen especial incluye la palabra “urgente” en la descripción, la prioridad se eleva a alta. El resto queda a cargo del operador, que sigue la siguiente referencia:

Tipo de solicitud	Prioridad por defecto	Se eleva cuando…
Certificado	Baja	Se necesita para un trámite con fecha (media) o que vence en pocos días (alta).
Equivalencia	Media	La inscripción a materias depende de ella y está por cerrar (alta). Si es parcial o para un cuatrimestre futuro, puede bajar a baja.
Baja de materia	Media	Se acerca la fecha límite para darla de baja (alta).
Examen especial	Media	La mesa está próxima o hay superposición de fechas (alta).
Otro	Baja	La consulta afecta la cursada actual (media).

Los datos de prueba del sistema fueron cargados siguiendo este criterio.

Tecnología utilizada

El sistema está desarrollado en Java 21, sin dependencias externas, con IntelliJ IDEA Community Edition como entorno de desarrollo. La interacción es por consola, mediante un menú numerado. La persistencia se resuelve con archivos de texto en formato CSV (campos separados por punto y coma) dentro de la carpeta data. El control de versiones se lleva con Git, y el repositorio está alojado en GitHub.

Para las pruebas se utilizaron datos sintéticos almacenados en tres archivos CSV: estudiantes.csv, responsables.csv y solicitudes.csv. El conjunto de datos incluye 5 estudiantes, 4 responsables y 8 solicitudes académicas, con distintos tipos, prioridades, estados e historial de seguimiento. Esto permite probar las principales operaciones del sistema con diferentes situaciones.

Versión inicial

La versión base del sistema está identificada en el repositorio con el **tag estado-inicial**.

Repositorio: https://github.com/bks999/tpi-solicitudes.git

Para ejecutarlo, se abre el proyecto en IntelliJ con el directorio de trabajo en la raíz del proyecto (donde se encuentra la carpeta data) y se ejecuta la clase Main.

Evidencia de funcionamiento

Menú principal

```
===== SISTEMA DE SOLICITUDES ACADEMICAS =====
1. Registrar solicitud
2. Cambiar estado de solicitud
3. Asignar responsable
4. Buscar / filtrar solicitudes
5. Generar reporte
6. Listar todas las solicitudes
0. Salir
Opcion:
```

Registro de una solicitud nueva

```
===== SISTEMA DE SOLICITUDES ACADEMICAS =====
1. Registrar solicitud
2. Cambiar estado de solicitud
3. Asignar responsable
4. Buscar / filtrar solicitudes
5. Generar reporte
6. Listar todas las solicitudes
0. Salir
Opcion: 1
Legajo del estudiante: 10234
Tipo de solicitud:
1. Certificado
2. Equivalencia
3. Baja de materia
4. Mesa de examen especial
5. Otro
Opcion: 2
Prioridad (1=baja, 2=media, 3=alta): 2
Descripcion: Equivalencias para inscripcion a materias
Solicitud registrada con ID 9
```

UTN
TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

Cambio de estado

```
===== SISTEMA DE SOLICITUDES ACADEMICAS =====
1. Registrar solicitud
2. Cambiar estado de solicitud
3. Asignar responsable
4. Buscar / filtrar solicitudes
5. Generar reporte
6. Listar todas las solicitudes
0. Salir
Opcion: 2
ID de la solicitud: 8
Estado actual: PENDIENTE
Nuevo estado (PENDIENTE, EN_REVISION, APROBADA, RECHAZADA, CERRADA): EN_REVISION
Estado actualizado correctamente
```

Asignación de responsable

```
===== SISTEMA DE SOLICITUDES ACADEMICAS =====
1. Registrar solicitud
2. Cambiar estado de solicitud
3. Asignar responsable
4. Buscar / filtrar solicitudes
5. Generar reporte
6. Listar todas las solicitudes
0. Salir
Opcion: 3
ID de la solicitud: 5
Responsables disponibles:
1 - Laura Gomez (BEDELIA)
2 - Diego Sosa (SECRETARIA_ACADEMICA)
3 - Paula Herrera (DEPARTAMENTO_ALUMNOS)
4 - Martin Castro (BEDELIA)
ID del responsable: 2
Responsable Diego Sosa asignado a la solicitud 5
```

UTN
TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

Búsqueda por un criterio

```
===== SISTEMA DE SOLICITUDES ACADEMICAS =====
1. Registrar solicitud
2. Cambiar estado de solicitud
3. Asignar responsable
4. Buscar / filtrar solicitudes
5. Generar reporte
6. Listar todas las solicitudes
0. Salir
Opcion: 4
Buscar por: 1. Estado  2. Tipo  3. Prioridad  4. Legajo  5. Responsable
Opcion: 3
Prioridad (1-3): 3
#3 | 2026-10-06 | BAJA_MATERIA | Prioridad: ALTA | Estado: APROBADA | Estudiante: Ferreyra, Julieta (11020) | Responsable: Paula Herrera
    Necesito dar de baja Matematica II, es urgente por superposicion horaria
#6 | 2026-10-18 | EXAMEN_ESPECIAL | Prioridad: ALTA | Estado: EN_REVISION | Estudiante: Rios, Martina (10234) | Responsable: Martin Castro
    Pido mesa especial urgente por superposicion con otra mesa
2 resultado(s)
```

Listado completo

```
===== SISTEMA DE SOLICITUDES ACADEMICAS =====
1. Registrar solicitud
2. Cambiar estado de solicitud
3. Asignar responsable
4. Buscar / filtrar solicitudes
5. Generar reporte
6. Listar todas las solicitudes
0. Salir
Opcion: 6
#1 | 2026-10-01 | CERTIFICADO | Prioridad: BAJA | Estado: PENDIENTE | Estudiante: Rios, Martina (10234) | Responsable: Sin asignar
    Solicito certificado de alumno regular para presentar en el trabajo
#2 | 2026-10-03 | EQUIVALENCIA | Prioridad: MEDIA | Estado: EN_REVISION | Estudiante: Benitez, Tomas (10587) | Responsable: Diego Sosa
    Pido equivalencia de Programacion I aprobada en otra universidad
#3 | 2026-10-06 | BAJA_MATERIA | Prioridad: ALTA | Estado: APROBADA | Estudiante: Ferreyra, Julieta (11020) | Responsable: Paula Herrera
    Necesito dar de baja Matematica II, es urgente por superposicion horaria
#4 | 2026-10-10 | EXAMEN_ESPECIAL | Prioridad: MEDIA | Estado: RECHAZADA | Estudiante: Medina, Lucas (11345) | Responsable: Laura Gomez
    Solicito mesa especial de Base de Datos por viaje laboral
#5 | 2026-10-15 | OTRO | Prioridad: BAJA | Estado: PENDIENTE | Estudiante: Acosta, Camila (11789) | Responsable: Diego Sosa
    Consulta sobre cambio de comision para el segundo cuatrimestre
#6 | 2026-10-18 | EXAMEN_ESPECIAL | Prioridad: ALTA | Estado: EN_REVISION | Estudiante: Rios, Martina (10234) | Responsable: Martin Castro
    Pido mesa especial urgente por superposicion con otra mesa
#7 | 2026-09-20 | CERTIFICADO | Prioridad: MEDIA | Estado: CERRADA | Estudiante: Benitez, Tomas (10587) | Responsable: Laura Gomez
    Certificado de materias aprobadas para tramite de beca
#8 | 2026-10-25 | EQUIVALENCIA | Prioridad: BAJA | Estado: EN_REVISION | Estudiante: Ferreyra, Julieta (11020) | Responsable: Diego Sosa
    Equivalencia parcial de Ingles I cursado en instituto
#9 | 2026-11-06 | EQUIVALENCIA | Prioridad: MEDIA | Estado: PENDIENTE | Estudiante: Rios, Martina (10234) | Responsable: Sin asignar
    Equivalencias para inscripcion a materias
```

UTN
TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

Reporte generado

```
===== SISTEMA DE SOLICITUDES ACADEMICAS =====
1. Registrar solicitud
2. Cambiar estado de solicitud
3. Asignar responsable
4. Buscar / filtrar solicitudes
5. Generar reporte
6. Listar todas las solicitudes
0. Salir
Opcion: 5
========== REPORTE DE SOLICITUDES ==========
Fecha: 2026-11-06
Total de solicitudes: 9

-- Por estado --
Pendientes: 3
En revision: 3
Aprobadas: 1
Rechazadas: 1
Cerradas: 1

-- Por tipo --
Certificados: 2
Equivalencias: 3
Bajas de materia: 1
Examenes especiales: 2
Otros: 1

-- Por responsable --
Laura Gomez (BEDELIA): 2
Diego Sosa (SECRETARIA_ACADEMICA): 3
Paula Herrera (DEPARTAMENTO_ALUMNOS): 1
Martin Castro (BEDELIA): 1
Sin asignar: 2

Prioridad promedio: 1.0
Solicitudes de prioridad alta abiertas: 2

Reporte guardado en data/reporte.txt
```

2. Calidad externa e interna

Calidad externa

La calidad externa se refiere a cómo percibe el sistema quien lo utiliza, independientemente de cómo esté construido internamente. Para evaluarla, se recorrieron las seis operaciones del menú desde el rol del operador administrativo, utilizando datos válidos y realizando distintas pruebas con entradas incorrectas.

Resultados de las pruebas:

- **Registro de solicitudes:** se verificó que es posible registrar una solicitud para un estudiante existente, indicando el tipo, la prioridad y una descripción. Al finalizar, el sistema asigna un ID y confirma el registro mediante un mensaje en consola.

```
===== SISTEMA DE SOLICITUDES ACADEMICAS =====
1. Registrar solicitud
2. Cambiar estado de solicitud
3. Asignar responsable
4. Buscar / filtrar solicitudes
5. Generar reporte
6. Listar todas las solicitudes
0. Salir
Opcion: 1
Legajo del estudiante: 10234
Tipo de solicitud:
1. Certificado
2. Equivalencia
3. Baja de materia
4. Mesa de examen especial
5. Otro
Opcion: 2
Prioridad (1=baja, 2=media, 3=alta): 2
Descripcion: Equivalencias para inscripcion a materias
Solicitud registrada con ID 9
```

- **Validación de reglas de negocio:** se comprobó que una solicitud no puede pasar al estado `EN_REVISION` si no tiene un responsable asignado.

```
===== SISTEMA DE SOLICITUDES ACADEMICAS =====
1. Registrar solicitud
2. Cambiar estado de solicitud
3. Asignar responsable
4. Buscar / filtrar solicitudes
5. Generar reporte
6. Listar todas las solicitudes
0. Salir
Opcion: 2
ID de la solicitud: 5
Estado actual: PENDIENTE
Nuevo estado (PENDIENTE, EN_REVISION, APROBADA, RECHAZADA, CERRADA): EN_REVISION
Error: no se puede pasar a revision sin responsable asignado
```

- También se comprobó la elevación automática de prioridad cuando una solicitud de examen especial contiene la palabra «urgente».

```
===== SISTEMA DE SOLICITUDES ACADEMICAS =====
1. Registrar solicitud
2. Cambiar estado de solicitud
3. Asignar responsable
4. Buscar / filtrar solicitudes
5. Generar reporte
6. Listar todas las solicitudes
0. Salir
Opcion: 1
Legajo del estudiante: 11234
Tipo de solicitud:
1. Certificado
2. Equivalencia
3. Baja de materia
4. Mesa de examen especial
5. Otro
Opcion: 4
Prioridad (1=baja, 2=media, 3=alta): 1
Descripcion: Solicitud de mesa de examen de la materia Metodologia II urgente
Aviso: la prioridad se elevo a ALTA por tratarse de un tramite urgente.
Solicitud registrada con ID 14
```

- **Registro de nuevos estudiantes:** cuando se ingresa un legajo que no existe, el sistema permite registrar los datos del nuevo estudiante y continuar con la creación de la solicitud.

```
===== SISTEMA DE SOLICITUDES ACADEMICAS =====
1. Registrar solicitud
2. Cambiar estado de solicitud
3. Asignar responsable
4. Buscar / filtrar solicitudes
5. Generar reporte
6. Listar todas las solicitudes
0. Salir
Opcion: 1
Legajo del estudiante: 11234
El estudiante no existe, se va a registrar.
Nombre: Sol
Apellido: Yoon
Email: sol@gmail.com
Carrera: Programacion
Tipo de solicitud:
1. Certificado
2. Equivalencia
3. Baja de materia
4. Mesa de examen especial
5. Otro
Opcion:
```

- **Validación de entradas incorrectas:** ante el ingreso de una opción no contemplada en el menú o un tipo de dato inválido, el sistema muestra un mensaje de error y permite volver al menú principal.

```
===== SISTEMA DE SOLICITUDES ACADEMICAS =====
1. Registrar solicitud
2. Cambiar estado de solicitud
3. Asignar responsable
4. Buscar / filtrar solicitudes
5. Generar reporte
6. Listar todas las solicitudes
0. Salir
Opcion: 7
Opcion invalida
```

```
===== SISTEMA DE SOLICITUDES ACADEMICAS =====
1. Registrar solicitud
2. Cambiar estado de solicitud
3. Asignar responsable
4. Buscar / filtrar solicitudes
5. Generar reporte
6. Listar todas las solicitudes
0. Salir
Opcion: d
Opcion invalida
```

- **Manejo de datos inválidos**: Ante el ingreso de un valor no numérico durante la búsqueda por prioridad, el sistema genera una excepción `NumberFormat`

`Exception` y finaliza inesperadamente. Por lo tanto, no siempre logra recuperarse de entradas inválidas.

- **Generación de reportes:** El sistema genera reportes con información estadística, pero presenta un error en el cálculo del promedio de prioridades. Con ocho solicitudes, informa 1,0 cuando el valor correcto es 1,875.

- **Ejecución desde otro directorio:** durante las pruebas se detectó que, al ejecutar el programa desde una carpeta distinta a la raíz del proyecto, el sistema no encuentra la carpeta `data` y muestra el mensaje «Error al cargar solicitudes». Aunque el menú continúa funcionando, algunas operaciones, como listar las solicitudes, no pueden realizarse porque los datos no se cargaron. El mensaje tampoco permite identificar claramente el origen del problema.

En general, el sistema cumple con las funcionalidades principales en condiciones normales de uso. Sin embargo, presenta limitaciones en el tratamiento de entradas inválidas, la ubicación de los archivos de datos, que afecta la disponibilidad de la información cuando el programa se ejecuta desde otro directorio, y la precisión de los resultados.

Esta evaluación permite identificar aspectos del comportamiento del sistema que pueden mejorarse.

Calidad interna

La calidad interna se relaciona con la organización del código, la distribución de responsabilidades, el nivel de acoplamiento entre sus componentes y la facilidad para comprender, mantener y modificar el sistema.

A partir de la revisión del código fuente, se identificaron algunos aspectos que afectan estas características.

En primer lugar, la clase SistemaSolicitudes concentra gran parte de la funcionalidad del programa. Además de gestionar el menú y la interacción con el operador, contiene lógica de negocio, operaciones de búsqueda, generación de reportes y lectura y escritura de archivos. Esta concentración de responsabilidades dificulta localizar y modificar una funcionalidad sin afectar otras partes del sistema.

También se observa que las clases Solicitud y Responsable exponen sus atributos mediante campos públicos. Esto permite acceder y modificar directamente sus datos desde otras clases, reduciendo el control sobre la información y dificultando el mantenimiento de las reglas de negocio.

Otro aspecto es que la persistencia se encuentra integrada directamente en la clase principal. La lectura y escritura de los archivos CSV no está separada de la lógica que

administra las solicitudes, lo que genera un mayor acoplamiento y dificulta cambiar el mecanismo de almacenamiento en el futuro.

Por otra parte, algunos métodos concentran varias operaciones y presentan una extensión considerable. Esto dificulta seguir el flujo de ejecución, comprender las responsabilidades de cada bloque y realizar modificaciones de manera localizada.

Finalmente, se identificaron situaciones en las que el manejo de entradas y excepciones podría mejorarse. Por ejemplo, algunas conversiones de datos pueden producir excepciones si el operador introduce un valor inesperado.

En conjunto, estos aspectos no impiden necesariamente que el sistema cumpla sus funciones habituales, pero representan dificultades para su mantenimiento, comprensión y evolución. Por este motivo, se considera conveniente realizar una refactorización que mejore la distribución de responsabilidades, el encapsulamiento y la legibilidad del código.

3. Diagnóstico de deuda técnica y code smells

A partir del análisis del código fuente de la versión inicial, se identificaron distintos problemas de diseño e implementación que pueden dificultar el mantenimiento y la evolución del sistema.

A continuación, se describen los principales *code smells* y debilidades técnicas detectados, junto con su ubicación, impacto y prioridad de corrección.

3.1. Concentración de responsabilidades (God Class)

- **Evidencia y ubicación:** la clase SistemaSolicitudes concentra distintas tareas, como la interacción con el usuario, la gestión de solicitudes, las búsquedas, la generación de reportes y la lectura y escritura de archivos.
- **Impacto:** la concentración de funcionalidades dificulta comprender la estructura de la clase y localizar el código correspondiente a cada operación.
- **Riesgo de mantenimiento:** modificar una funcionalidad puede afectar otras responsabilidades que también se encuentran dentro de la misma clase.
- **Prioridad de corrección:** Alta.

3.2. Métodos extensos que realizan varias tareas (Long Method)

- **Evidencia y ubicación:** los métodos registrarSolicitud() y reporte() de la clase SistemaSolicitudes concentran varias operaciones dentro de un mismo bloque. Por ejemplo, registrarSolicitud() combina la solicitud de datos al usuario, las validaciones

y la creación de registros. Por su parte, reporte() realiza recorridos y cálculos para generar la información estadística.
- **Impacto:** la cantidad de operaciones dificulta seguir el flujo de ejecución y comprender qué tarea realiza cada parte del método.
- **Riesgo de mantenimiento:** realizar modificaciones o corregir errores requiere revisar bloques extensos de código, aumentando la posibilidad de afectar otras operaciones.
- **Prioridad de corrección:** Alta.

3.3. Atributos públicos que permiten modificar datos sin control (Public Fields)

- **Evidencia y ubicación:** las clases Solicitud y Responsable exponen todos sus atributos públicamente, lo que permite acceder a ellos y modificarlos directamente desde otras clases, a diferencia de Estudiante que posee getters y setters.
- **Impacto:** se reduce el encapsulamiento, ya que los objetos no controlan completamente cómo se modifican sus propios datos.
- **Riesgo de mantenimiento:** cualquier parte del programa puede alterar los atributos sin respetar las validaciones o reglas de negocio que deberían aplicarse.
- **Prioridad de corrección:** Alta.

3.4. Acoplamiento entre la lógica de negocio y la persistencia (Tight Coupling)

- **Evidencia y ubicación:** en la clase SistemaSolicitudes se encuentran tanto las operaciones relacionadas con la gestión de solicitudes como los métodos encargados de cargar y guardar información en archivos CSV.
- **Impacto:** las operaciones del sistema dependen directamente del mecanismo utilizado para almacenar los datos, lo que dificulta separar y comprender cada responsabilidad.
- **Riesgo de mantenimiento:** si en el futuro se necesita cambiar el almacenamiento, por ejemplo, reemplazar los archivos CSV por una base de datos, será necesario modificar partes de la clase que también contienen lógica de negocio.
- **Prioridad de corrección:** Alta.

3.5. Uso de tipos primitivos para representar conceptos del dominio (Primitive Obsession)

- **Evidencia y ubicación:**
    - Los estados de las solicitudes (`"PENDIENTE"`, `"EN_REVISION"`, `"CERRADA"`, entre otros) se representan mediante cadenas de texto y se comparan en distintos métodos, como `cambiarEstado()`, `asignarResponsable()` y `reporte()`.



    - Los tipos de solicitud (`"CERTIFICADO"`, `"EQUIVALENCIA"`, `"EXAMEN_ESPECIAL"`, entre otros) también se representan mediante cadenas que se repiten en diferentes operaciones.
    - La prioridad se representa mediante números enteros (`1`, `2` y `3`). Su significado se establece mediante validaciones y comparaciones distribuidas en distintos métodos.
    - El valor `0` para `idResponsable` se utiliza para indicar que una solicitud no tiene responsable asignado.
- **Impacto:** el significado de estos valores depende del conocimiento previo del código. Además, su utilización directa puede generar inconsistencias ante errores de escritura o el uso de valores no contemplados.
- **Riesgo de mantenimiento:** al incorporar o modificar estados, tipos de solicitud o prioridades, es necesario revisar los distintos lugares donde se utilizan estos valores. Esto aumenta la posibilidad de omitir alguna comparación o validación.
- **Prioridad de corrección:** Media.

3.6. Funcionalidad pendiente o incompleta (Incomplete implementation)

- **Evidencia y ubicación:** En la clase `SistemaSolicitudes` se encuentra el método `calcularDiasAbierta(Solicitud s)`, que conserva un comentario `TODO` y devuelve siempre `0`. No se encontraron invocaciones a este método en el código.
- **Impacto:** Actualmente no afecta el funcionamiento observable del sistema, ya que el método no es utilizado.
- **Riesgo de mantenimiento:** Si se incorpora su uso en futuras funcionalidades sin completar su implementación, podría proporcionar información incorrecta sobre los días que una solicitud permanece abierta.
- **Prioridad de corrección:** Media.

3.7. Manejo insuficiente de entradas y excepciones (Poor Error Handling)

- **Evidencia y ubicación:**
    - En el método `buscar()` de `SistemaSolicitudes`, la búsqueda por prioridad utiliza `Integer.parseInt()` sin controlar adecuadamente la excepción `NumberFormatException`. Al ingresar un valor no numérico, el programa finaliza inesperadamente (CP-07).
    - En la búsqueda por responsable existe un bloque `catch` vacío que captura la excepción, pero no informa el error ni solicita un nuevo ingreso. Como consecuencia, al ingresar `abc`, el sistema ejecuta la búsqueda como si se hubiera ingresado `0` y devuelve las solicitudes sin responsable asignado (CP-12).
- **Impacto:** los datos ingresados incorrectamente no reciben un tratamiento adecuado y pueden provocar la interrupción del programa o resultados inesperados.

- **Riesgo de mantenimiento:** la falta de un manejo uniforme de las excepciones dificulta identificar y corregir errores en las operaciones de búsqueda.
- **Prioridad de corrección:** Alta.

3.8. Elementos sin utilización efectiva (Dead Code)

- **Evidencia y ubicación:**
  - En `SistemaSolicitudes`, la variable `flag` está declarada, pero no se utiliza.
  - Al final de la clase `SistemaSolicitudes` se encuentra el método `eliminarSolicitud(int id)` completamente comentado, por lo que no puede ejecutarse.
- **Impacto:** estos elementos no aportan funcionalidad al sistema y generan código innecesario.
- **Riesgo de mantenimiento:** la presencia de código sin uso puede dificultar la lectura y comprensión de la clase, especialmente si se confunde con funcionalidades activas.
- **Prioridad de corrección:** Baja.

3.9. División entera en el cálculo del promedio

- **Evidencia y ubicación:** en el método reporte() de la clase SistemaSolicitudes, el cálculo del promedio utiliza sumaPrioridad / lista.size(). Como ambos operandos son enteros, Java realiza una división entera antes de asignar el resultado a la variable `double`.
- **Impacto:** el resultado pierde la parte decimal y puede mostrar un promedio incorrecto. Por ejemplo, con las ocho solicitudes de los archivos CSV, el promedio esperado es 1,875, pero el cálculo devuelve 1,0.
- **Riesgo de mantenimiento:** los reportes pueden presentar información estadística inexacta y generar interpretaciones equivocadas.
- **Prioridad de corrección:** Alta.

3.10. Lista extensa de parámetros (Long Parameter List)

- **Evidencia y ubicación:** el método `crearSolicitud()` de `SistemaSolicitudes` recibe nueve parámetros para construir un objeto `Solicitud`, incluyendo identificadores, datos descriptivos, prioridad, estado y fechas.
- **Impacto:** la cantidad de parámetros dificulta la lectura de la llamada al método y hace menos claro el significado de cada argumento.
- **Riesgo de mantenimiento:** al modificar los datos necesarios para crear una solicitud, es necesario revisar la firma del método y todas sus llamadas, aumentando la posibilidad de errores.
- **Prioridad de corrección:** Media.

3.11. Nombres poco descriptivos (Poor Naming)

- **Evidencia y ubicación:** en `SistemaSolicitudes` se utilizan identificadores poco descriptivos, como `lista2` para almacenar estudiantes, `resp` para responsables y `aux()` para un método que normaliza textos. También aparecen abreviaturas como `leg`, `nom` y `pr`.
- **Impacto:** estos nombres dificultan comprender rápidamente el propósito de las variables y métodos, especialmente para alguien que no conoce previamente el código.
- **Riesgo de mantenimiento:** la falta de claridad en los identificadores puede generar confusiones al modificar el código y dificultar su lectura por parte de otros desarrolladores.
- **Prioridad de corrección:** Media.

4. Aplicación de SOLID y Clean Code

5. Refactorings aplicados con evidencia antes y después

6. Pruebas y verificación del comportamiento

6.1. Pruebas sobre la versión inicial

Para evaluar el comportamiento del sistema antes de realizar las refactorizaciones, se definieron distintos casos de prueba orientados a verificar las funcionalidades principales, las reglas de negocio y el tratamiento de situaciones excepcionales.

Cada caso establece un objetivo, las condiciones necesarias para ejecutarlo, el resultado esperado, el resultado obtenido y las evidencias correspondientes.

CP-01. Carga inicial de datos

- **Objetivo:** verificar que el sistema cargue correctamente la información almacenada en los archivos CSV.
- **Precondiciones:** disponer de los archivos `solicitudes.csv`, `estudiantes.csv` y `responsables.csv` originales.
- **Ejecución:** iniciar el programa y consultar las solicitudes registradas.
- **Resultado esperado:** cargar los registros existentes desde los archivos CSV.
- **Resultado obtenido:** se visualizaron correctamente las 8 solicitudes, incluyendo fechas, tipos, prioridades, estados, estudiantes y responsables asignados.
- **Estado**: Correcto
- **Evidencia:** captura del listado completo de solicitudes.

```
#1 | 2026-10-01 | CERTIFICADO | Prioridad: BAJA | Estado: PENDIENTE | Estudiante: Rios, Martina (10234) | Responsable: Sin asignar
    Solicito certificado de alumno regular para presentar en el trabajo
#2 | 2026-10-03 | EQUIVALENCIA | Prioridad: MEDIA | Estado: EN_REVISION | Estudiante: Benitez, Tomas (10587) | Responsable: Diego Sosa
    Pido equivalencia de Programacion I aprobada en otra universidad
#3 | 2026-10-06 | BAJA_MATERIA | Prioridad: ALTA | Estado: APROBADA | Estudiante: Ferreyra, Julieta (11020) | Responsable: Paula Herrera
    Necesito dar de baja Matematica II, es urgente por superposicion horaria
#4 | 2026-10-10 | EXAMEN_ESPECIAL | Prioridad: MEDIA | Estado: RECHAZADA | Estudiante: Medina, Lucas (11345) | Responsable: Laura Gomez
    Solicito mesa especial de Base de Datos por viaje laboral
#5 | 2026-10-15 | OTRO | Prioridad: BAJA | Estado: PENDIENTE | Estudiante: Acosta, Camila (11789) | Responsable: Sin asignar
    Consulta sobre cambio de comision para el segundo cuatrimestre
#6 | 2026-10-18 | EXAMEN_ESPECIAL | Prioridad: ALTA | Estado: EN_REVISION | Estudiante: Rios, Martina (10234) | Responsable: Martin Castro
    Pido mesa especial urgente por superposicion con otra mesa
#7 | 2026-09-20 | CERTIFICADO | Prioridad: MEDIA | Estado: CERRADA | Estudiante: Benitez, Tomas (10587) | Responsable: Laura Gomez
    Certificado de materias aprobadas para tramite de beca
#8 | 2026-10-25 | EQUIVALENCIA | Prioridad: BAJA | Estado: PENDIENTE | Estudiante: Ferreyra, Julieta (11020) | Responsable: Diego Sosa
    Equivalencia parcial de Ingles I cursado en instituto
```

CP-02. Registro de una solicitud para un estudiante existente

- **Objetivo:** comprobar que se pueda registrar una nueva solicitud para un estudiante previamente registrado.
- **Precondiciones:** disponer de los datos iniciales y utilizar un legajo existente.
- **Ejecución:** ingresar los datos solicitados, seleccionando un estudiante existente y completando el tipo, la prioridad y la descripción.
- **Resultado esperado:** registrar una solicitud para un estudiante existente y asignarle un identificador.
- **Resultado obtenido:** el sistema registró la solicitud para Martina Ríos (10234) y confirmó la operación con el mensaje «Solicitud registrada con ID 9».
- **Estado**: Correcto
- **Evidencia:** captura del mensaje de confirmación y del registro generado.

```
===== SISTEMA DE SOLICITUDES ACADEMICAS =====
1. Registrar solicitud
2. Cambiar estado de solicitud
3. Asignar responsable
4. Buscar / filtrar solicitudes
5. Generar reporte
6. Listar todas las solicitudes
0. Salir
Opcion: 1
Legajo del estudiante: 10234
Tipo de solicitud:
1. Certificado
2. Equivalencia
3. Baja de materia
4. Mesa de examen especial
5. Otro
Opcion: 1
Prioridad (1=baja, 2=media, 3=alta): 1
Descripcion: Solicitud de prueba para verificar el registro
Solicitud registrada con ID 9
```

CP-03. Registro de un estudiante nuevo

- **Objetivo:** verificar que el sistema permita registrar un estudiante que todavía no se encuentra en los archivos.
- **Precondiciones:** utilizar un legajo que no exista en `estudiantes.csv`.
- **Ejecución:** ingresar un nuevo legajo, completar los datos del estudiante y continuar con el registro de una solicitud.
- **Resultado esperado:** si el legajo ingresado no existe, el sistema debe solicitar los datos del estudiante, registrarlo y permitir continuar con el alta de la solicitud.
- **Resultado obtenido:** El sistema detectó que el estudiante no existía, solicitó sus datos y permitió registrar la solicitud. Luego, al listar las solicitudes, se visualizaron correctamente los registros con ID 9 y 10, incluyendo los datos de ambos estudiantes y sus respectivas solicitudes.
- **Estado**: Correcto
- **Evidencia:** Captura de la consola donde se observe el registro del estudiante y la confirmación de la solicitud, seguido por la captura del listado donde se observan las solicitudes 9 y 10.

```
Opcion: 1
Legajo del estudiante: 12001
El estudiante no existe, se va a registrar.
Nombre: Valentina
Apellido: Perez
Email: valentina.perez@mail.com
Carrera: TUPaD
Tipo de solicitud:
1. Certificado
2. Equivalencia
3. Baja de materia
4. Mesa de examen especial
5. Otro
Opcion: 5
Prioridad (1=baja, 2=media, 3=alta): 2
Descripcion: Solicitud de prueba para estudiante nuevo
Solicitud registrada con ID 10
```

```
#1 | 2026-10-01 | CERTIFICADO | Prioridad: BAJA | Estado: PENDIENTE | Estudiante: Rios, Martina (10234) | Responsable: Sin asignar
    Solicito certificado de alumno regular para presentar en el trabajo
#2 | 2026-10-03 | EQUIVALENCIA | Prioridad: MEDIA | Estado: EN_REVISION | Estudiante: Benitez, Tomas (10587) | Responsable: Diego Sosa
    Pido equivalencia de Programacion I aprobada en otra universidad
#3 | 2026-10-06 | BAJA_MATERIA | Prioridad: ALTA | Estado: APROBADA | Estudiante: Ferreyra, Julieta (11020) | Responsable: Paula Herrera
    Necesito dar de baja Matematica II, es urgente por superposicion horaria
#4 | 2026-10-10 | EXAMEN_ESPECIAL | Prioridad: MEDIA | Estado: RECHAZADA | Estudiante: Medina, Lucas (11345) | Responsable: Laura Gomez
    Solicito mesa especial de Base de Datos por viaje laboral
#5 | 2026-10-15 | OTRO | Prioridad: BAJA | Estado: PENDIENTE | Estudiante: Acosta, Camila (11789) | Responsable: Sin asignar
    Consulta sobre cambio de comision para el segundo cuatrimestre
#6 | 2026-10-18 | EXAMEN_ESPECIAL | Prioridad: ALTA | Estado: EN_REVISION | Estudiante: Rios, Martina (10234) | Responsable: Martin Castro
    Pido mesa especial urgente por superposicion con otra mesa
#7 | 2026-09-20 | CERTIFICADO | Prioridad: MEDIA | Estado: CERRADA | Estudiante: Benitez, Tomas (10587) | Responsable: Laura Gomez
    Certificado de materias aprobadas para tramite de beca
#8 | 2026-10-25 | EQUIVALENCIA | Prioridad: BAJA | Estado: PENDIENTE | Estudiante: Ferreyra, Julieta (11020) | Responsable: Diego Sosa
    Equivalencia parcial de Ingles I cursado en instituto
#9 | 2026-11-06 | CERTIFICADO | Prioridad: BAJA | Estado: PENDIENTE | Estudiante: Rios, Martina (10234) | Responsable: Sin asignar
    Solicitud de prueba para verificar el registro
#10 | 2026-11-06 | OTRO | Prioridad: MEDIA | Estado: PENDIENTE | Estudiante: Perez, Valentina (12001) | Responsable: Sin asignar
    Solicitud de prueba para estudiante nuevo
```

CP-04. Validación de la asignación de responsable

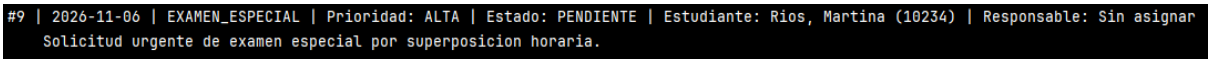
- **Objetivo:** comprobar que se respete la regla de negocio que impide pasar una solicitud a `EN_REVISION` sin un responsable asignado.
- **Precondiciones:** utilizar la solicitud con ID 1, que inicialmente se encuentra en estado `PENDIENTE` y no tiene responsable asignado.
- **Ejecución:** intentar cambiar su estado a `EN_REVISION`.
- **Resultado esperado:** El sistema debe impedir que una solicitud sin responsable asignado pase al estado `EN_REVISION` y mostrar un mensaje de error.
- **Resultado obtenido:** Al intentar cambiar la solicitud 1 de `PENDIENTE` a `EN_REVISION`, el sistema rechazó la operación y mostró el mensaje: «Error: no se puede pasar a revision sin responsable asignado».
- **Estado:** Correcto
- **Evidencia:** Captura de la consola con el intento de cambio y el mensaje de error.

```
===== SISTEMA DE SOLICITUDES ACADEMICAS =====
1. Registrar solicitud
2. Cambiar estado de solicitud
3. Asignar responsable
4. Buscar / filtrar solicitudes
5. Generar reporte
6. Listar todas las solicitudes
0. Salir
Opcion: 2
ID de la solicitud: 1
Estado actual: PENDIENTE
Nuevo estado (PENDIENTE, EN_REVISION, APROBADA, RECHAZADA, CERRADA): EN_REVISION
Error: no se puede pasar a revision sin responsable asignado
```

CP-05. Aumento automático de prioridad para solicitudes urgentes de examen especial

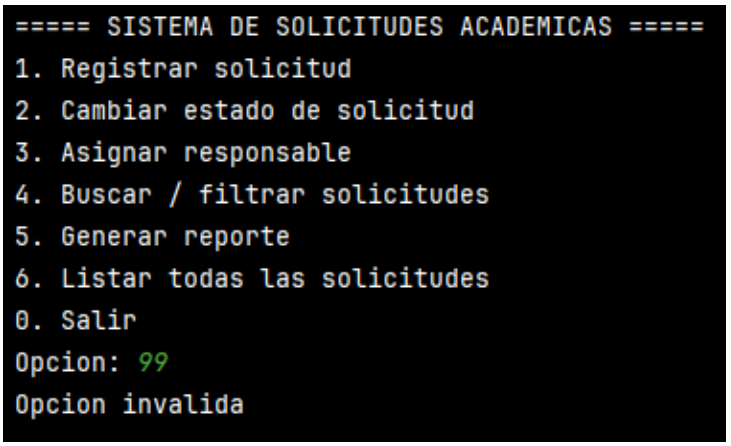
- **Objetivo:** verificar la aplicación de la regla de elevación de prioridad ante una solicitud urgente.
- **Precondiciones:** contar con un estudiante registrado.
- **Ejecución:** registrar una solicitud de tipo `EXAMEN_ESPECIAL`, incluyendo la palabra «urgente» en su descripción.
- **Resultado esperado:** Cuando se registra una solicitud de tipo `EXAMEN_ESPECIAL` cuya descripción contiene la palabra «urgente», el sistema debe elevar automáticamente su prioridad a `ALTA`.
- **Resultado obtenido:** Se ingresó una solicitud de examen especial con prioridad inicial baja y una descripción que contenía la palabra «urgente». El sistema mostró el aviso de elevación de prioridad y confirmó el registro con el ID 9.
- **Estado:** Correcto
- **Evidencia:** Captura de la consola donde se observe la prioridad inicial, el aviso y la confirmación del registro. Luego, captura de la solicitud en lista, con prioridad Alta.

```
Opcion: 1
Legajo del estudiante: 10234
Tipo de solicitud:
1. Certificado
2. Equivalencia
3. Baja de materia
4. Mesa de examen especial
5. Otro
Opcion: 4
Prioridad (1=baja, 2=media, 3=alta): 1
Descripcion: Solicitud urgente de examen especial por superposicion horaria.
Aviso: la prioridad se elevo a ALTA por tratarse de un tramite urgente.
Solicitud registrada con ID 9
```



CP-06. Validación de opciones incorrectas en el menú

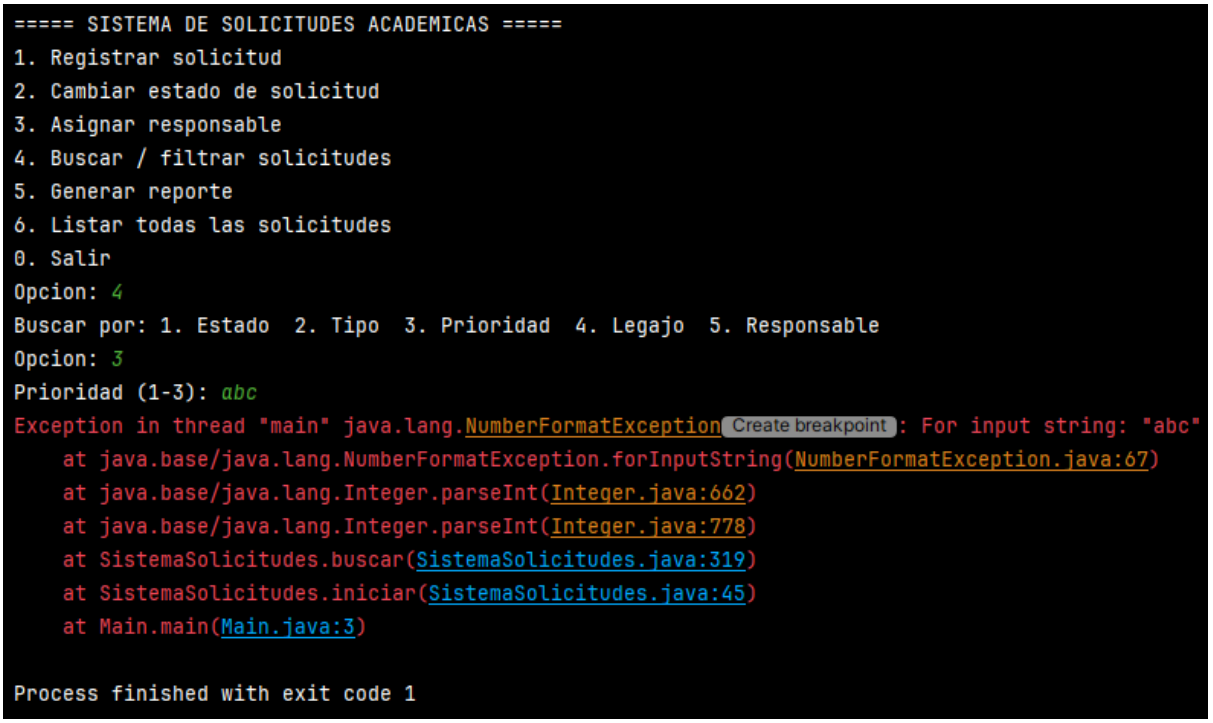
- **Objetivo:** comprobar cómo responde el sistema ante una opción de menú no contemplada.
- **Precondiciones:** programa iniciado y menú principal disponible.
- **Ejecución:** ingresar una opción inexistente.
- **Resultado esperado:** Si el usuario ingresa una opción que no pertenece al menú, el sistema debe rechazarla y mostrar un mensaje de error, sin ejecutar ninguna operación.
- **Resultado obtenido:** Al ingresar `99`, el sistema mostró el mensaje «Opcion invalida» y no ejecutó ninguna operación.
- **Estado:** Correcto
- **Evidencia:** Captura de la consola con el ingreso de `99` y el mensaje recibido.



CP-07. Búsqueda mediante una prioridad no numérica

- **Objetivo:** evaluar el tratamiento de entradas incorrectas durante la búsqueda por prioridad.
- **Precondiciones:** programa iniciado y opción de búsqueda disponible.
- **Ejecución:** seleccionar la búsqueda por prioridad e ingresar un valor no numérico, por ejemplo, `abc`.
- **Resultado esperado:** Si el usuario ingresa un valor que no es numérico al buscar por prioridad, el sistema debe mostrar un mensaje de error y permitir continuar utilizando el programa.
- **Resultado obtenido:** Al ingresar `abc`, se produjo una excepción `NumberFormatException` en el método `buscar()` de la clase `SistemaSolicitudes` (línea 319). El programa finalizó su ejecución con código de salida 1.
- **Estado**: Fallido

- **Evidencia:** Captura de la consola donde se observe el ingreso de `abc`, la excepción y la finalización del programa.

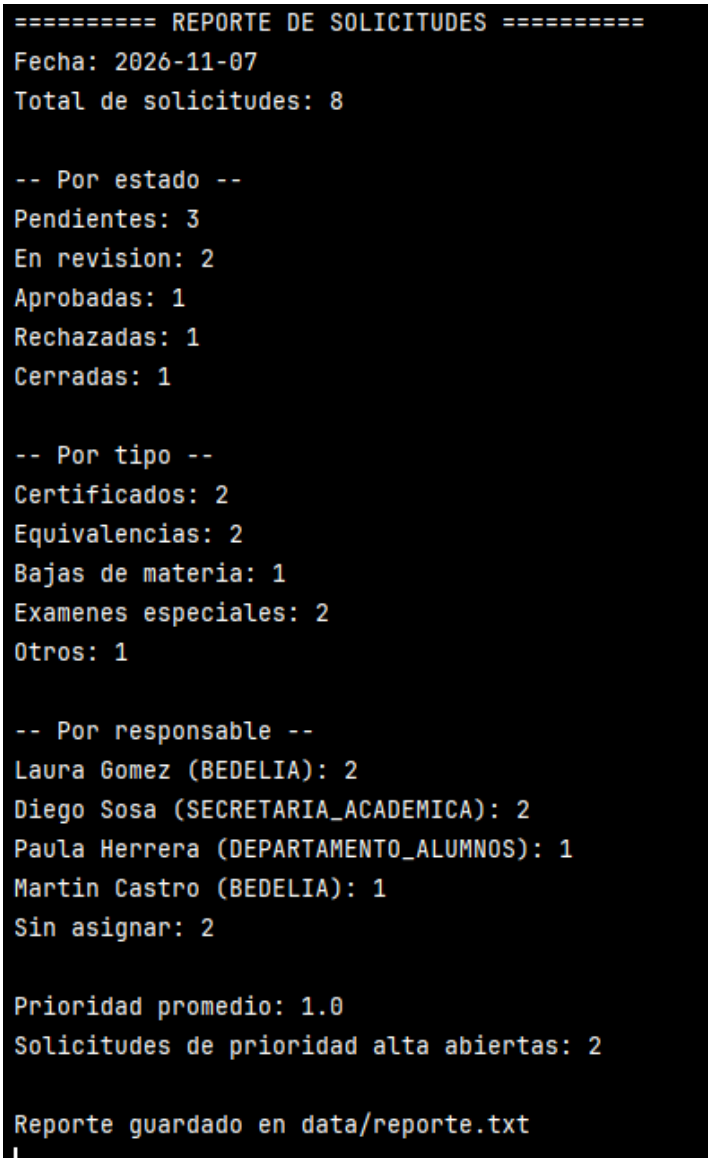


Esta prueba demuestra:

- Falta de validación de la entrada del usuario.
- Ausencia de manejo de `NumberFormatException`.
- Interrupción inesperada del programa ante un dato inválido.

CP-08. Cálculo del promedio de prioridades

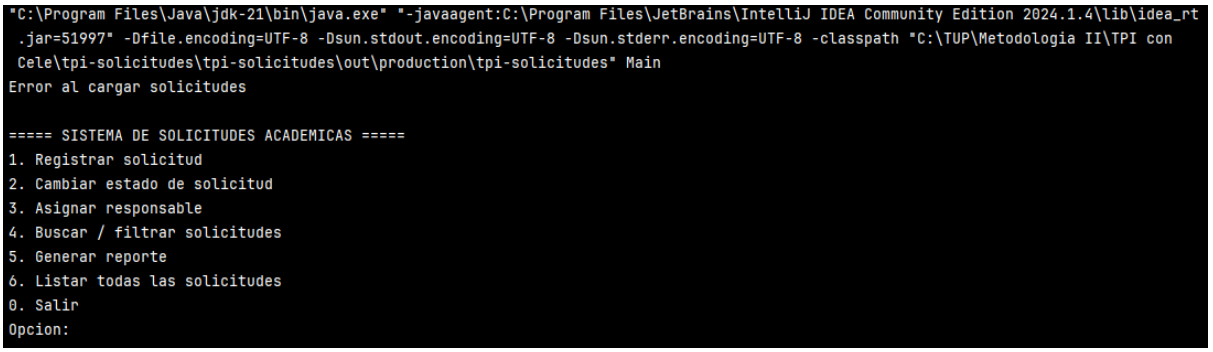
- **Objetivo:** verificar que el reporte estadístico calcule correctamente el promedio de prioridades.
- **Precondiciones:** utilizar los ocho registros originales de `solicitudes.csv`.
- **Ejecución:** generar el reporte y observar el promedio de prioridades.
- **Resultado esperado:** El sistema debe calcular el promedio de prioridades de las ocho solicitudes originales. La suma de las prioridades es 15, por lo que el promedio esperado es 1,875.
- **Resultado obtenido:** El reporte informó un promedio de prioridad de 1.0, aunque el cálculo correcto es 1,875.
- **Estado:** Fallido
- **Evidencia:** Captura del reporte donde se observen las ocho solicitudes y el promedio informado.



CP-09. Ejecución desde un directorio diferente

- **Objetivo:** comprobar el comportamiento del sistema cuando se inicia desde una ubicación distinta a la raíz del proyecto.
- **Precondiciones:** disponer del proyecto y sus archivos de datos en su ubicación habitual.
- **Ejecución:** iniciar el programa utilizando como directorio de ejecución otra carpeta.
- **Resultado esperado:** El sistema debería localizar los archivos de datos y cargar las solicitudes correctamente, independientemente del directorio desde el que se ejecute.
- **Resultado obtenido:** al ejecutar el programa desde un directorio de trabajo diferente al del proyecto (`C:\TUP`), el sistema mostró el mensaje `Error al cargar solicitudes`. Aunque el menú principal se presentó correctamente, los archivos CSV no pudieron cargarse.

- **Estado**: Incorrecto
- **Evidencia:** captura de la consola donde se observa el mensaje de error y la presentación del menú principal.



Esta prueba demuestra:

- Dependencia de rutas relativas para acceder a los archivos CSV.
- Falta de control adecuado ante la ausencia de archivos.
- Mensajes de error poco claros para identificar el problema.

CP-10. Asignación de responsable con área correspondiente

- **Objetivo:** comprobar que el sistema permita asignar un responsable cuya área coincida con el tipo de solicitud.
- **Precondiciones:**
  - Solicitud 1: `CERTIFICADO`, estado `PENDIENTE`, sin responsable asignado.
  - Responsable 1: Laura Gomez, área `BEDELIA`.
- **Ejecución:** ejecutar el programa y seleccionar la opción **Asignar responsable**. Luego, ingresar el ID de la solicitud (1) y el ID del responsable (1).
- **Resultado esperado:** el sistema debe permitir la asignación sin solicitar una confirmación por diferencia de áreas y mostrar un mensaje indicando que la asignación se realizó correctamente.
- **Resultado obtenido:** Se asignó correctamente a Laura Gómez a la solicitud 1. El sistema no solicitó confirmación adicional.
- **Estado**: Correcto
- **Evidencia:** captura de la consola con la asignación realizada.




```
===== SISTEMA DE SOLICITUDES ACADEMICAS =====
1. Registrar solicitud
2. Cambiar estado de solicitud
3. Asignar responsable
4. Buscar / filtrar solicitudes
5. Generar reporte
6. Listar todas las solicitudes
0. Salir
Opcion: 3
ID de la solicitud: 1
Responsables disponibles:
1 - Laura Gomez (BEDELIA)
2 - Diego Sosa (SECRETARIA_ACADEMICA)
3 - Paula Herrera (DEPARTAMENTO_ALUMNOS)
4 - Martin Castro (BEDELIA)
ID del responsable: 1
Responsable Laura Gomez asignado a la solicitud 1
```

CP-11. Asignación de responsable de un área diferente

- **Objetivo:** comprobar que el sistema advierta cuando se intenta asignar un responsable cuya área no coincide con el área correspondiente al tipo de solicitud y solicite confirmación para continuar.
- **Precondiciones:**
  - Solicitud 1: `CERTIFICADO`, estado `PENDIENTE`, sin responsable asignado.
  - Responsable 2: Diego Sosa, área SECRETARIA_ACADEMICA.
- **Ejecución:** ejecutar el programa y seleccionar la opción **Asignar responsable**. Luego, ingresar el ID de la solicitud (1) y el ID del responsable (2). Finalmente, responder `s` a la solicitud de confirmación.
- **Resultado esperado:** el sistema debe advertir que el tipo de solicitud `CERTIFICADO` corresponde al área `BEDELIA` y solicitar confirmación antes de realizar la asignación.
- **Resultado obtenido:** al seleccionar a Diego Sosa (SECRETARIA_ACADEMICA), el sistema advirtió que CERTIFICADO corresponde a BEDELIA y solicitó confirmación. Al responder `s`, permitió realizar la asignación.
- **Estado**: Correcto
- **Evidencia:** captura de la consola con la advertencia, la confirmación y la asignación realizada.

```
===== SISTEMA DE SOLICITUDES ACADEMICAS =====
1. Registrar solicitud
2. Cambiar estado de solicitud
3. Asignar responsable
4. Buscar / filtrar solicitudes
5. Generar reporte
6. Listar todas las solicitudes
0. Salir
Opcion: 3
ID de la solicitud: 1
Responsables disponibles:
1 - Laura Gomez (BEDELIA)
2 - Diego Sosa (SECRETARIA_ACADEMICA)
3 - Paula Herrera (DEPARTAMENTO_ALUMNOS)
4 - Martin Castro (BEDELIA)
ID del responsable: 2
Atencion: el tipo CERTIFICADO corresponde al area BEDELIA. Asignar igual? (s/n): s
Responsable Diego Sosa asignado a la solicitud 1
```

CP-12. Búsqueda de solicitudes por responsable

- **Objetivo:** comprobar cómo responde el sistema cuando se ingresa un dato no válido al buscar solicitudes por responsable.
- **Precondiciones:** disponer de los datos iniciales cargados, incluyendo solicitudes con y sin responsable asignado.
- **Ejecución:** ejecutar el programa, seleccionar la opción Buscar / filtrar solicitudes, elegir la búsqueda por responsable (opción 5) e ingresar `abc` como ID del responsable.
- **Resultado esperado:** el sistema debe validar que el ID ingresado sea numérico y mostrar un mensaje indicando que el valor no es válido, sin ejecutar una búsqueda incorrecta.
- **Resultado obtenido:** el sistema no informó que el dato ingresado fuera inválido. En su lugar, mostró las solicitudes 1 y 5, ambas sin responsable asignado, como resultado de la búsqueda.
- **Estado**: Incorrecto
- **Evidencia:** captura de la consola donde se observa el dato ingresado y las dos solicitudes devueltas.

```
===== SISTEMA DE SOLICITUDES ACADEMICAS =====
1. Registrar solicitud
2. Cambiar estado de solicitud
3. Asignar responsable
4. Buscar / filtrar solicitudes
5. Generar reporte
6. Listar todas las solicitudes
0. Salir
Opcion: 4
Buscar por: 1. Estado  2. Tipo  3. Prioridad  4. Legajo  5. Responsable
Opcion: 5
ID responsable (0 = sin asignar): abc
#1 | 2026-10-01 | CERTIFICADO | Prioridad: BAJA | Estado: PENDIENTE | Estudiante: Rios, Martina (10234) | Responsable: Sin asignar
    Solicito certificado de alumno regular para presentar en el trabajo
#5 | 2026-10-15 | OTRO | Prioridad: BAJA | Estado: PENDIENTE | Estudiante: Acosta, Camila (11789) | Responsable: Sin asignar
    Consulta sobre cambio de comision para el segundo cuatrimestre
2 resultado(s)
```

UTN
TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

7. Code review profesional

8. Conclusiones y aprendizaje

9. Anexos y enlaces